UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

Sistemas Distribuidos

Clase 10 — Tracing, métricas, logs y debugging en producción

Departamento de Computación
Ingeniería Informática – Comisión A – Pilar
Equipo docente – 2026

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

El problema operacional y los silos

Sistemas Distribuidos

Después de diseñar, hay que operar

Ya vimos RPC, retries, consistencia, transacciones y logs. Vimos componentes y cómo diseñar un sistema.

Después de diseñar, hay que operar

Ya vimos RPC, retries, consistencia, transacciones y logs. Vimos componentes y cómo diseñar un sistema.
¿Qué es lo que aparece una vez que el sistema está desplegado en producción?

Después de diseñar, hay que operar

Ya vimos RPC, retries, consistencia, transacciones y logs. Vimos componentes y cómo diseñar un sistema.
¿Qué es lo que aparece una vez que el sistema está desplegado en producción? **Problemas**

Después de diseñar, hay que operar

Ya vimos RPC, retries, consistencia, transacciones y logs. Vimos componentes y cómo diseñar un sistema.
¿Qué es lo que aparece una vez que el sistema está desplegado en producción? **Problemas**



Después de diseñar, hay que operar

Ya vimos RPC, retries, consistencia, transacciones y logs. Vimos componentes y cómo diseñar un sistema.
¿Qué es lo que aparece una vez que el sistema está desplegado en producción? **Problemas**



El desafío real

No se trata solo de que el sistema opere correctamente, sino de poder entenderlo mientras opera. Por ejemplo, qué está fallando y por qué.

Un solo servidor vs. Múltiples servidores en red

En un único servidor (o monolito en producción):

- **Stack trace completo:** el hilo de ejecución vive en el mismo espacio de memoria; ante un fallo, el runtime explica la causalidad.
- **Reloj físico único:** el hardware serializa los logs locales con orden real.
- **Llamadas en memoria:** una llamada entre funciones toma nanosegundos. El tiempo que tarda el sistema se puede explicar mirando un solo lugar.

Un solo servidor vs. Múltiples servidores en red

Al pasar a múltiples servidores:

- **Stack trace desaparece:** el usuario final solo ve 4xx o 5xx.
- ***Clock drift*:** logs de distintas instancias no permiten seguir el orden causa $\rightarrow$ efecto.
- **Tiempos ocultos:** la latencia empieza a depender de otros servicios y de la red.
- **Fallas combinatorias:** cada nodo se ve "sano" en sus métricas locales, pero la latencia se acumula.

Un solo servidor vs. Múltiples servidores en red

Al pasar a múltiples servidores:

- **Stack trace desaparece:** el usuario final solo ve 4xx o 5xx.
- ***Clock drift*:** logs de distintas instancias no permiten seguir el orden causa $\rightarrow$ efecto.
- **Tiempos ocultos:** la latencia empieza a depender de otros servicios y de la red.
- **Fallas combinatorias:** cada nodo se ve "sano" en sus métricas locales, pero la latencia se acumula.

Por lo tanto

La observabilidad no es un agregado opcional al final del desarrollo. Es una **propiedad arquitectónica estructural** del sistema.

¿Por qué monitorear?

- **Detección temprana:** Sin monitoreo, la primera señal de fallo suele ser un usuario quejándose.
- **Diagnóstico:** Hay problemas que no se resuelven de manera automática y es necesario informar a un humano: un disco que se está llenando, un certificado SSL por vencer, un servicio que degrada lentamente.
- **Decisiones informadas:** ¿Cuánto mejoró el hit rate de memcache al agregar un nodo? Sin métricas, esa respuesta es una apuesta.
- **Contención de daño:** Un sistema que detecta su propia degradación puede activar mecanismos de resiliencia antes de que el daño se propague en cascada.
- **Aprendizaje continuo:** Los datos de monitoreo alimentan postmortems blameless, experimentos de caos (Chaos Engineering) y mejoras continuas de diseño arquitectónico.

La propuesta clásica: “Los Tres Pilares”

Tradicionalmente, la industria ha presentado la observabilidad como un sistema de tres señales complementarias (los tres pilares):

- **Métricas:** Agregaciones numéricas y tendencias temporales globales para evaluar la salud general y disparar alertas.
- **Logs:** Registros detallados de eventos individuales que ocurren en el código de cada servicio.
- **Trazas:** El recorrido topológico de una solicitud individual a través de los múltiples saltos de red.

La propuesta clásica: “Los Tres Pilares”

Tradicionalmente, la industria ha presentado la observabilidad como un sistema de tres señales complementarias (los tres pilares):

- **Métricas:** Agregaciones numéricas y tendencias temporales globales para evaluar la salud general y disparar alertas.
- **Logs:** Registros detallados de eventos individuales que ocurren en el código de cada servicio.
- **Trazas:** El recorrido topológico de una solicitud individual a través de los múltiples saltos de red.

La promesa

Si recolectamos las tres señales de forma exhaustiva, tendremos visibilidad suficiente para operar y entender el sistema en producción.

La propuesta clásica: “Los Tres Pilares”

Cada una de las tres señales demandó requerimientos técnicos radicalmente distintos:

Métricas: Escritura numérica masiva y agregación temporal en memoria → Bases de Series Temporales (*TSDB*, ej: Prometheus).

La propuesta clásica: "Los Tres Pilares"

Cada una de las tres señales demandó requerimientos técnicos radicalmente distintos:

Métricas: Escritura numérica masiva y agregación temporal en memoria → Bases de Series Temporales (*TSDB*, ej: Prometheus).

Logs: Búsqueda sobre texto libre no estructurado → Motores de indexación invertida (*Lucene*, ej: Elasticsearch).

La propuesta clásica: "Los Tres Pilares"

Cada una de las tres señales demandó requerimientos técnicos radicalmente distintos:

Métricas: Escritura numérica masiva y agregación temporal en memoria → Bases de Series Temporales (*TSDB*, ej: Prometheus).

Logs: Búsqueda sobre texto libre no estructurado → Motores de indexación invertida (*Lucene*, ej: Elasticsearch).

Trazas: Reconstrucción de árboles y grafos acíclicos → Almacenamiento especializado de spans (*Dapper*, ej: Jaeger).

La propuesta clásica: "Los Tres Pilares"

Cada una de las tres señales demandó requerimientos técnicos radicalmente distintos:

Métricas: Escritura numérica masiva y agregación temporal en memoria → Bases de Series Temporales (*TSDB*, ej: Prometheus).

Logs: Búsqueda sobre texto libre no estructurado → Motores de indexación invertida (*Lucene*, ej: Elasticsearch).

Trazas: Reconstrucción de árboles y grafos acíclicos → Almacenamiento especializado de spans (*Dapper*, ej: Jaeger).

El resultado técnico

Cada solución evolucionó de forma aislada: con su propio agente en el host, su propio formato de datos y su propio protocolo de red.

El antipatrón de “Las Tres Pestañas”

En la práctica, la divergencia tecnológica se trasladó a la pantalla del operador durante un incidente:



Three Browser Tabs

- Pestaña 1: Dashboards de métricas (Grafana).
- Pestaña 2: Buscador de logs (Elasticsearch / Kibana).
- Pestaña 3: Herramienta de trazas aisladas (Jaeger).

El antipatrón de “Las Tres Pestañas”

En la práctica, la divergencia tecnológica se trasladó a la pantalla del operador durante un incidente:



Three Browser Tabs

- Pestaña 1: Dashboards de métricas (Grafana).
- Pestaña 2: Buscador de logs (Elasticsearch / Kibana).
- Pestaña 3: Herramienta de trazas aisladas (Jaeger).

El problema operativo: La información vive inconexa. El ser humano se ve forzado a ser el “puente manual”, memorizando IDs, alineando marcas de tiempo a ojo e intentando adivinar correlaciones entre tres herramientas a las 4 AM.

La solución arquitectónica: “La Trenza Única de Datos”

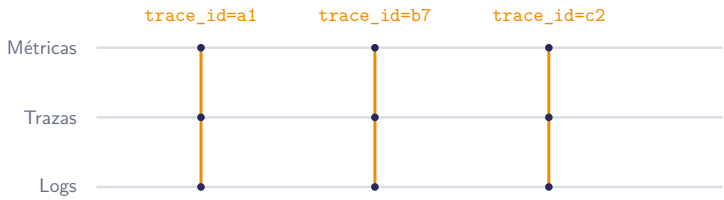
Single Braid of Data (OpenTelemetry / OTLP)

Las tres señales conservan su identidad y propósito, pero viajan y se almacenan **entrelazadas mediante contexto compartido**.

La solución arquitectónica: “La Trenza Única de Datos”

Single Braid of Data (OpenTelemetry / OTLP)

Las tres señales conservan su identidad y propósito, pero viajan y se almacenan **entrelazadas mediante contexto compartido**.

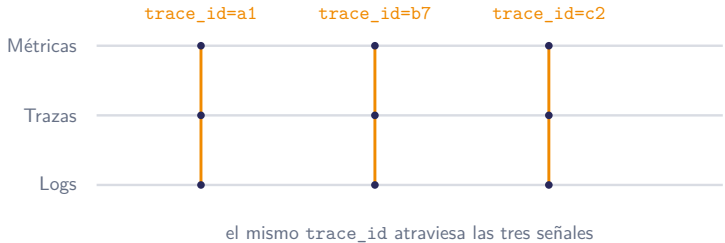


el mismo `trace_id` atraviesa las tres señales

La solución arquitectónica: “La Trenza Única de Datos”

Single Braid of Data (OpenTelemetry / OTLP)

Las tres señales conservan su identidad y propósito, pero viajan y se almacenan **entrelazadas mediante contexto compartido**.



Adaptado de Young & Parker, *Learning OpenTelemetry* (O’Reilly, 2024), Fig. 1-3.

Monitoreo y observabilidad no son lo mismo

Monitoreo ¿Está funcionando el sistema? Chequea fallas que alguien **anticipó**: ¿hay memoria? ¿responde el ping? ¿subió el *error rate*? Son *known-unknowns*: sabés qué preguntar, no sabés la respuesta.

Monitoreo y observabilidad no son lo mismo

Monitoreo ¿Está funcionando el sistema? Chequea fallas que alguien **anticipó**: ¿hay memoria? ¿responde el ping? ¿subió el *error rate*? Son *known-unknowns*: sabés qué preguntar, no sabés la respuesta.

Observabilidad ¿Por qué está pasando **esto**, que nadie previó? Son *unknown-unknowns*: ni siquiera sabías que había que preguntarlo.

Monitoreo y observabilidad no son lo mismo

Monitoreo ¿Está funcionando el sistema? Chequea fallas que alguien **anticipó**: ¿hay memoria? ¿responde el ping? ¿subió el *error rate*? Son *known-unknowns*: sabés qué preguntar, no sabés la respuesta.

Observabilidad ¿Por qué está pasando **esto**, que nadie previó? Son *unknown-unknowns*: ni siquiera sabías que había que preguntarlo.

La definición formal (Teoría de Control)

Un sistema es observable si su estado interno puede inferirse a partir del conocimiento de sus salidas externas.

Monitoreo y observabilidad no son lo mismo

Monitoreo ¿Está funcionando el sistema? Chequea fallas que alguien **anticipó**: ¿hay memoria? ¿responde el ping? ¿subió el *error rate*? Son *known-unknowns*: sabés qué preguntar, no sabés la respuesta.

Observabilidad ¿Por qué está pasando **esto**, que nadie previó? Son *unknown-unknowns*: ni siquiera sabías que había que preguntarlo.

La definición formal (Teoría de Control)

Un sistema es observable si su estado interno puede inferirse a partir del conocimiento de sus salidas externas.

Llevado a software: explicar cualquier estado del sistema haciendo preguntas nuevas, **sin desplegar código** para agregar más logs. Si para responder hay que deployar, no era observable.

Kálmán (1960); Majors et al. (2022); Young & Parker (2024).

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

El pegamento: Contexto e Instrumentación

Sistemas Distribuidos

El hilo de la trenza: Hard Context vs. Soft Context

Esas verticales naranjas de la trenza tienen nombre: **el Contexto**. Metadatos que describen la relación entre operaciones y telemetría.

El hilo de la trenza: Hard Context vs. Soft Context

Esas verticales naranjas de la trenza tienen nombre: **el Contexto**. Metadatos que describen la relación entre operaciones y telemetría.

- **Hard Context:** Identificador único por solicitud que los servicios propagan entre sí (`trace_id`, `span_id`, cabeceras W3C). Permite relacionar **explícitamente** mediciones que tienen relación causal. Define la **forma** del sistema.

El hilo de la trenza: Hard Context vs. Soft Context

- **Soft Context:** Atributos que cada instrumento adjunta a la medición (`customer_id`, hostname, *timestamp*). **Puede** haber relación causal, pero no está garantizada. Define las **dimensiones** por las que podés cortar los datos.

El hilo de la trenza: Hard Context vs. Soft Context

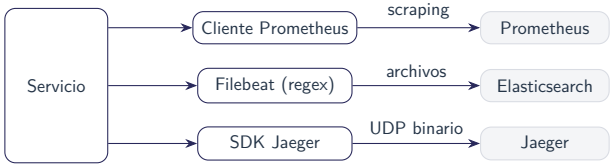
- **Soft Context:** Atributos que cada instrumento adjunta a la medición (`customer_id`, hostname, *timestamp*). **Puede** haber relación causal, pero no está garantizada. Define las **dimensiones** por las que podés cortar los datos.

No es uno o el otro

El soft context es lo que deja descubrir que la falla correlaciona con `tenant_id=450`. El problema es el soft context *sin* hard context: ahí el tiempo queda como única dimensión compartida y alguien tiene que alinear *timestamps* a ojo. Eso son las tres pestañas.

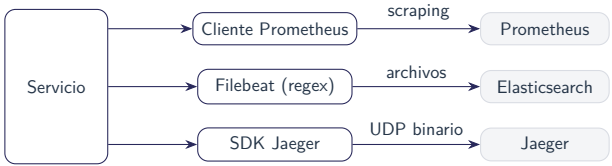
Antes de OpenTelemetry: un agente por señal

Volvamos al problema de los agentes incompatibles: la trenza no se sostiene si cada señal viaja por su propio cable.



Antes de OpenTelemetry: un agente por señal

Volvamos al problema de los agentes incompatibles: la trenza no se sostiene si cada señal viaja por su propio cable.



El costo

Memoria y CPU del host compitiendo con la aplicación, y acoplamiento: cambiar de backend obliga a reescribir la instrumentación.

OpenTelemetry: un SDK, un protocolo, un Collector



1. **SDK único:** el código se instrumenta contra una API neutral; no sabe a qué backend van los datos.

OpenTelemetry: un SDK, un protocolo, un Collector



1. **SDK único:** el código se instrumenta contra una API neutral; no sabe a qué backend van los datos.
2. **OTLP (el protocolo):** un solo canal para métricas, logs y trazas, enviados en *batches*. Por defecto `http/protobuf`.

OpenTelemetry: un SDK, un protocolo, un Collector



1. **SDK único:** el código se instrumenta contra una API neutral; no sabe a qué backend van los datos.
2. **OTLP (el protocolo):** un solo canal para métricas, logs y trazas, enviados en *batches*. Por defecto `http/protobuf`.
3. **Collector:** proceso aparte (*sidecar* o agente de nodo) que recibe, transforma, filtra o muestrea, y exporta a varios destinos por configuración.

OpenTelemetry: un SDK, un protocolo, un Collector



1. **SDK único:** el código se instrumenta contra una API neutral; no sabe a qué backend van los datos.
2. **OTLP (el protocolo):** un solo canal para métricas, logs y trazas, enviados en *batches*. Por defecto `http/protobuf`.
3. **Collector:** proceso aparte (*sidecar* o agente de nodo) que recibe, transforma, filtra o muestrea, y exporta a varios destinos por configuración.

Clave:

OpenTelemetry es la capa de transporte, no la de almacenamiento: no se ata a ningún proveedor.

De la teoría al código: Estrategia en 2 Capas

¿Cómo implementamos esto?

1. **Capa 1: Infraestructura (Auto-instrumentación)** Middlewares HTTP/gRPC y agentes que capturan el esqueleto de red y propagan el **Hard Context** (cabeceras W3C) automáticamente.

De la teoría al código: Estrategia en 2 Capas

¿Cómo implementamos esto?

1. **Capa 1: Infraestructura (Auto-instrumentación)** Middlewares HTTP/gRPC y agentes que capturan el esqueleto de red y propagan el **Hard Context** (cabeceras W3C) automáticamente.
2. **Capa 2: Dominio (Instrumentación Manual)** El desarrollador inyecta atributos de negocio de alta cardinalidad (`user_id`, `cart_items`) en el contexto activo de la solicitud.

Correlación automática (I): el software hace de puente

En las tres pestañas, el puente era el operador. Con hard context compartido, el puente es un *JOIN*.

Contexto activo: al recibir la solicitud, el SDK guarda `trace_id` y `span_id` en el contexto del hilo (en Java, el `Context` de OTel sobre un `ThreadLocal`).

Correlación automática (I): el software hace de puente

En las tres pestañas, el puente era el operador. Con hard context compartido, el puente es un *JOIN*.

Contexto activo: al recibir la solicitud, el SDK guarda `trace_id` y `span_id` en el contexto del hilo (en Java, el `Context` de OTel sobre un `ThreadLocal`).

Log ↔ Traza: cada llamada al logger sale con esos IDs estampados, sin que el desarrollador los pase.

```
{"level":"error", "msg":"timeout DB", "trace_id":"a1", "span_id":"7f"}
```

Correlación automática (I): el software hace de puente

En las tres pestañas, el puente era el operador. Con hard context compartido, el puente es un *JOIN*.

Contexto activo: al recibir la solicitud, el SDK guarda `trace_id` y `span_id` en el contexto del hilo (en Java, el `Context` de OTel sobre un `ThreadLocal`).

Log ↔ Traza: cada llamada al logger sale con esos IDs estampados, sin que el desarrollador los pase.

```
{"level":"error", "msg":"timeout DB", "trace_id":"a1", "span_id":"7f"}
```

Resultado: el span y todos sus logs se unen por igualdad exacta de ID, no por cercanía de *timestamps*.

Correlación automática (II): de la métrica a la causa

Métrica $\leftrightarrow$ Traza (*Exemplars*): un *bucket* del histograma guarda el `trace_id` de una solicitud concreta. Clic en el pico de p99 en Grafana $\rightarrow$ se abre la traza que lo causó.

Correlación automática (II): de la métrica a la causa

Métrica $\leftrightarrow$ Traza (*Exemplars*): un *bucket* del histograma guarda el `trace_id` de una solicitud concreta. Clic en el pico de p99 en Grafana $\rightarrow$ se abre la traza que lo causó.

Análisis diferencial: como todas las señales comparten atributos, el motor compara su distribución entre solicitudes fallidas y exitosas, y aísla la dimensión culpable sin hipótesis previa.

Correlación automática (II): de la métrica a la causa

Métrica ↔ Traza (*Exemplars*): un *bucket* del histograma guarda el `trace_id` de una solicitud concreta. Clic en el pico de p99 en Grafana → se abre la traza que lo causó.

Análisis diferencial: como todas las señales comparten atributos, el motor compara su distribución entre solicitudes fallidas y exitosas, y aísla la dimensión culpable sin hipótesis previa.

Conexión con el diagnóstico

Esto es lo que va a hacer posible el Core Analysis Loop del final: métrica → dimensión → traza.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Las tres señales de la trenza

¿Qué responde cada señal?

Un sistema observable debe responder dos preguntas operativas:

1. **¿Qué está roto?** $\longrightarrow$ **Métricas agregadas** (salud y tendencias temporales globales).
2. **¿Dónde y por qué está roto?** $\longrightarrow$ **Trazas y Eventos Anchos** (causalidad distribuida y contexto de dominio).

¿Qué responde cada señal?

Un sistema observable debe responder dos preguntas operativas:

1. **¿Qué está roto?** $\longrightarrow$ **Métricas agregadas** (salud y tendencias temporales globales).
2. **¿Dónde y por qué está roto?** $\longrightarrow$ **Trazas y Eventos Anchos** (causalidad distribuida y contexto de dominio).

Principio rector

Una alerta que solo dice "error rate alto" es una alarma de incendio sin indicar la habitación. Las tres señales se complementan para aislar el fuego.

Señal 1: Métricas

Responde: *¿Qué componente o servicio está degradado?*

Qué es una métrica

Una representación numérica del estado del sistema durante el intervalo en que fue registrada: **nombre + etiquetas + una serie de pares (timestamp, valor)**.

Señal 1: Métricas

Responde: *¿Qué componente o servicio está degradado?*

Qué es una métrica

Una representación numérica del estado del sistema durante el intervalo en que fue registrada: **nombre + etiquetas + una serie de pares (timestamp, valor)**.

La propiedad que define todo: es *pre-agregada*

La pregunta se decide cuando **escribís** la métrica, no cuando la consultás. Los requests individuales que generaron ese número ya no existen. Es decir, no podés desagregar para ver detalle.

Señal 1: Métricas

Responde: *¿Qué componente o servicio está degradado?*

Qué es una métrica

Una representación numérica del estado del sistema durante el intervalo en que fue registrada: **nombre + etiquetas + una serie de pares (timestamp, valor)**.

La propiedad que define todo: es *pre-agregada*

La pregunta se decide cuando **escribís** la métrica, no cuando la consultás. Los requests individuales que generaron ese número ya no existen. Es decir, no podés desagregar para ver detalle.

- Una solicitud deja rastro en cientos de métricas y ninguna sabe de las otras: no hay hard context.

Las cuatro Golden Signals, una por una

Latencia Tiempo que toma atender una solicitud. Es bueno **separar la de éxito de la de error**: por ejemplo un HTTP 500 por caída de la DB podría responder rapidísimo y, promediado con el resto, *mejora* el número. Y un error lento es todavía peor que un error rápido.

Las cuatro Golden Signals, una por una

Latencia Tiempo que toma atender una solicitud. Es bueno **separar la de éxito de la de error**: por ejemplo un HTTP 500 por caída de la DB podría responder rapidísimo y, promediado con el resto, *mejora* el número. Y un error lento es todavía peor que un error rápido.

Tráfico Cuánta demanda recibe el sistema, en la unidad que corresponda: solicitudes por segundo en una API (rps), sesiones concurrentes en streaming, lecturas por segundo en una DB, etc. Mide *demanda*, no capacidad: si cae, puede ser buena noticia o que nadie logra entrar.

Las cuatro Golden Signals, una por una

Latencia Tiempo que toma atender una solicitud. Es bueno **separar la de éxito de la de error**: por ejemplo un HTTP 500 por caída de la DB podría responder rapidísimo y, promediado con el resto, *mejora* el número. Y un error lento es todavía peor que un error rápido.

Tráfico Cuánta demanda recibe el sistema, en la unidad que corresponda: solicitudes por segundo en una API (rps), sesiones concurrentes en streaming, lecturas por segundo en una DB, etc. Mide *demanda*, no capacidad: si cae, puede ser buena noticia o que nadie logra entrar.

Errores Tasa de solicitudes que fallan: por ejemplo, HTTP 5xx, HTTP 4xx con algún error de negocio.

Las cuatro Golden Signals, una por una

Latencia Tiempo que toma atender una solicitud. Es bueno **separar la de éxito de la de error**: por ejemplo un HTTP 500 por caída de la DB podría responder rapidísimo y, promediado con el resto, *mejora* el número. Y un error lento es todavía peor que un error rápido.

Tráfico Cuánta demanda recibe el sistema, en la unidad que corresponda: solicitudes por segundo en una API (rps), sesiones concurrentes en streaming, lecturas por segundo en una DB, etc. Mide *demanda*, no capacidad: si cae, puede ser buena noticia o que nadie logra entrar.

Errores Tasa de solicitudes que fallan: por ejemplo, HTTP 5xx, HTTP 4xx con algún error de negocio.

Saturación Qué tan lleno está el recurso más restringido (memoria, pool de conexiones de DB, CPU). Los sistemas se degradan *antes* del 100 % de utilización, y el alza de latencia es el **indicador adelantado**.

La latencia no es un número: es una distribución

Hasta ahora la tratamos como un escalar: “más consistencia global, más latencia”.

La latencia no es un número: es una distribución

Hasta ahora la tratamos como un escalar: “más consistencia global, más latencia”.

La falacia del promedio

99 usuarios tardan 10 ms y 1 usuario tarda 10 000 ms. El promedio es **110 ms**: un número que no describe a **nadie** —ni al que esperó 10 ms ni al que esperó 10 segundos—.

La latencia no es un número: es una distribución

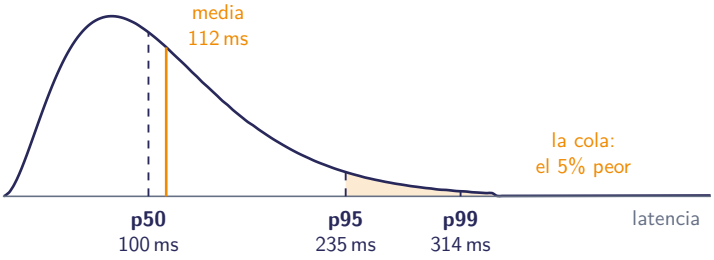
Hasta ahora la tratamos como un escalar: “más consistencia global, más latencia”.

La falacia del promedio

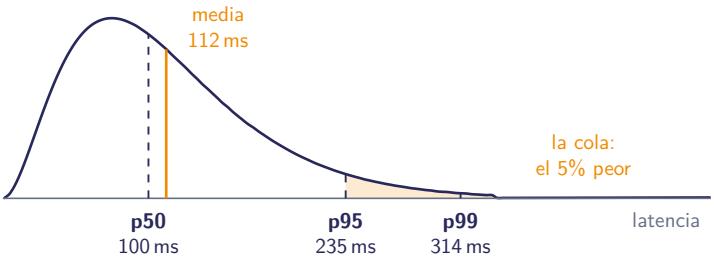
99 usuarios tardan 10 ms y 1 usuario tarda 10 000 ms. El promedio es **110 ms**: un número que no describe a **nadie** —ni al que esperó 10 ms ni al que esperó 10 segundos—.

Los percentiles no son exclusivos de las métricas: también se calculan sobre spans y eventos anchos, y ahí son *más* flexibles —un p99 de Prometheus sale de cubos de histograma que elegiste al instrumentar—.

Percentiles sobre una distribución de latencia real



Percentiles sobre una distribución de latencia real



- No es una campana: la latencia es **asimétrica**. Tiene piso físico y no tiene techo.
- Por eso la **media** cae a la derecha de la mediana, y el **p99 triplica al p50**.

El límite de las métricas: cardinalidad

Si la métrica es pre-agregada y perdimos el detalle, la pregunta natural es: **¿y por qué no le agrego más etiquetas?**

El límite de las métricas: cardinalidad

Si la métrica es pre-agregada y perdimos el detalle, la pregunta natural es: **¿y por qué no le agrego más etiquetas?**

Porque cada combinación de etiquetas es una serie temporal distinta

$$\text{series} = \text{métricas} \times \text{endpoints} \times \text{status} \times \mathbf{user_ids}$$

Una TSDB como Prometheus mantiene cada serie en memoria. Agregar `user_id` con 10 millones de usuarios **multiplica**, no suma.

El límite de las métricas: cardinalidad

Si la métrica es pre-agregada y perdimos el detalle, la pregunta natural es: **¿y por qué no le agrego más etiquetas?**

Porque cada combinación de etiquetas es una serie temporal distinta

$$\text{series} = \text{métricas} \times \text{endpoints} \times \text{status} \times \mathbf{user_ids}$$

Una TSDB como Prometheus mantiene cada serie en memoria. Agregar `user_id` con 10 millones de usuarios **multiplica**, no suma.

La regla

Alta cardinalidad en **spans y eventos anchos**. Nunca en métricas.

El límite de las métricas: cardinalidad

Si la métrica es pre-agregada y perdimos el detalle, la pregunta natural es: **¿y por qué no le agrego más etiquetas?**

Porque cada combinación de etiquetas es una serie temporal distinta

$$\text{series} = \text{métricas} \times \text{endpoints} \times \text{status} \times \mathbf{user_ids}$$

Una TSDB como Prometheus mantiene cada serie en memoria. Agregar `user_id` con 10 millones de usuarios **multiplica**, no suma.

La regla

Alta cardinalidad en **spans y eventos anchos**. Nunca en métricas.

Por eso la Capa 2 inyecta `user_id` en el span y no en un contador: el span describe *una* solicitud; la métrica, un agregado de todas.

Señal 2: Tracing Distribuido (DAG y Spans)

Responde: *¿Dónde estuvo la solicitud y en qué orden causal viajó?*

- **Trace:** Grafo Acíclico Dirigido (DAG) del viaje completo a través de los microservicios.
- **Span:** Unidad individual de trabajo delimitada en tiempo.

Señal 2: Tracing Distribuido (DAG y Spans)

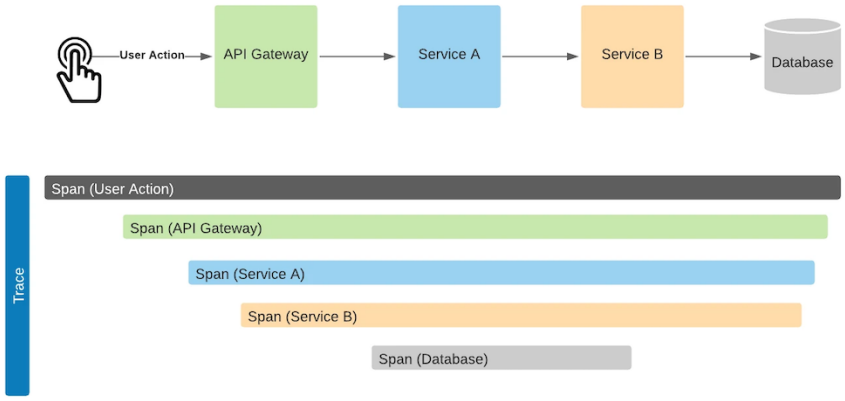
Responde: *¿Dónde estuvo la solicitud y en qué orden causal viajó?*

- **Trace:** Grafo Acíclico Dirigido (DAG) del viaje completo a través de los microservicios.
- **Span:** Unidad individual de trabajo delimitada en tiempo.

Campos esenciales de un Span:

`Trace ID` | `Span ID` | `Parent Span ID` | `Timestamp` | `Duration`

Un trace es esto



Arriba la topología, abajo la línea de tiempo: la misma solicitud vista de dos maneras. Cada barra **contiene** a la siguiente, y eso es por el `parent_span_id`.

Tracing y Causalidad: Lamport vs. Clock Drift

¿Por qué el timestamp de inicio no alcanza?

Por el *clock drift*, dos servidores desfasados por 50 ms registrarán eventos en orden inverso al real si los ordenamos por tiempo físico.

Tracing y Causalidad: Lamport vs. Clock Drift

¿Por qué el timestamp de inicio no alcanza?

Por el *clock drift*, dos servidores desfasados por 50 ms registrarán eventos en orden inverso al real si los ordenamos por tiempo físico.

- El campo `parent_span_id` implementa la relación ***happened-before* ($\rightarrow$) de Lamport**.
- Reconstruye el orden causal estricto sin depender de relojes físicos sincronizados.

Propagación, Collector y Muestreo

- **In-Band:** Cabeceras W3C (`traceparent`) viajan dentro de los mensajes de red. Si un microservicio olvida inyectarlas, la traza se corta.
- **Out-of-Band:** Envío asíncrono hacia el **OTel Collector** para buffering sin penalizar la latencia del usuario.

Propagación, Collector y Muestreo

- **In-Band:** Cabeceras W3C (`traceparent`) viajan dentro de los mensajes de red. Si un microservicio olvida inyectarlas, la traza se corta.
- **Out-of-Band:** Envío asíncrono hacia el **OTel Collector** para buffering sin penalizar la latencia del usuario.

Sampling (Trade-off de escala)

No es viable almacenar 10^9 trazas: se decide qué guardar al inicio (*head-based*) o al finalizar para retener errores y colas lentas (*tail-based*). Por ejemplo: guardar el 100 % de las trazas que tengan errores o alta latencia, y solo el 5 % del tráfico normal.

Señal 3: Eventos Estructurados Anchos (Wide Events)

Responde: *¿Por qué ocurrió el problema en este caso específico?*

- Logs de texto libre $\rightarrow$ inútiles a escala (regexes frágiles).
- Logs estructurados (JSON) $\rightarrow$ legibles por máquinas.

Señal 3: Eventos Estructurados Anchos (Wide Events)

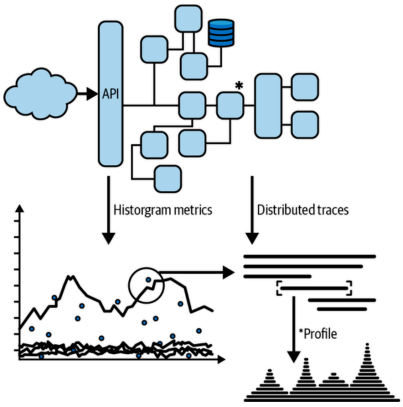
Responde: *¿Por qué ocurrió el problema en este caso específico?*

- Logs de texto libre $\rightarrow$ inútiles a escala (regexes frágiles).
- Logs estructurados (JSON) $\rightarrow$ legibles por máquinas.

Unidad de trabajo (Unit of Work)

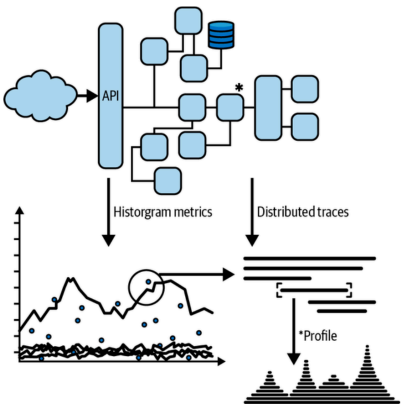
En lugar de emitir múltiples líneas sueltas, se inicializa un map en memoria al recibir la solicitud, se acumulan todos los atributos de negocio durante la ejecución y se emite un **único evento estructurado ancho** al finalizar.

Cómo se encadenan las tres señales



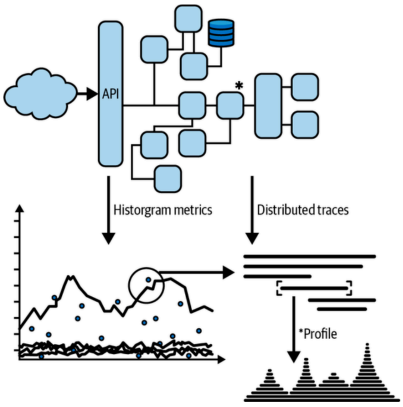
1. **Histograma de latencia** de la API: métrica agregada. Dice que *algo* se degradó.

Cómo se encadenan las tres señales



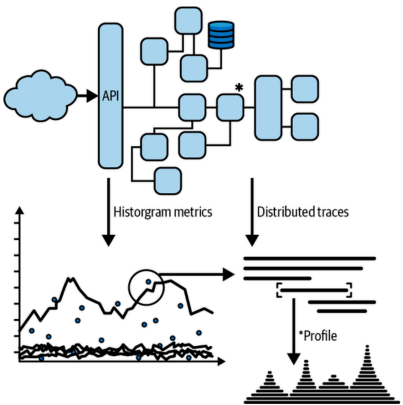
1. **Histograma de latencia** de la API: métrica agregada. Dice que *algo* se degradó.
2. **Exemplar**: el *bucket* del pico guarda el `trace_id` de una solicitud concreta. Es el salto de lo agregado a lo individual.

Cómo se encadenan las tres señales



1. **Histograma de latencia** de la API: métrica agregada. Dice que *algo* se degradó.
2. **Exemplar**: el *bucket* del pico guarda el `trace_id` de una solicitud concreta. Es el salto de lo agregado a lo individual.
3. **La traza**: en qué span se fue el tiempo.

Cómo se encadenan las tres señales



Young & Parker, *Learning OpenTelemetry* (O'Reilly, 2024), Fig. 2-4.

1. **Histograma de latencia** de la API: métrica agregada. Dice que *algo* se degradó.
2. **Exemplar**: el *bucket* del pico guarda el `trace_id` de una solicitud concreta. Es el salto de lo agregado a lo individual.
3. **La traza**: en qué span se fue el tiempo.
4. **El log o el *profile*** de ese span: el detalle por componente o por función.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Diagnóstico en producción

Sistemas Distribuidos

La causa raíz no está donde explota el síntoma

La cascada

1. Una base de datos aguas abajo se degrada por contención de bloqueos.
2. El servicio intermedio agota su pool de hilos encolando peticiones.
3. Los clientes aguas arriba reintentan automáticamente y **amplifican** el tráfico.
4. El API Gateway del borde empieza a devolver HTTP 504.

La causa raíz no está donde explota el síntoma

La cascada

1. Una base de datos aguas abajo se degrada por contención de bloqueos.
2. El servicio intermedio agota su pool de hilos encolando peticiones.
3. Los clientes aguas arriba reintentan automáticamente y **amplifican** el tráfico.
4. El API Gateway del borde empieza a devolver HTTP 504.

Lo que ve el usuario, y lo que dispara la alerta, es el paso 4. Lo que hay que arreglar es el 1.

La causa raíz no está donde explota el síntoma

La cascada

1. Una base de datos aguas abajo se degrada por contención de bloqueos.
2. El servicio intermedio agota su pool de hilos encolando peticiones.
3. Los clientes aguas arriba reintentan automáticamente y **amplifican** el tráfico.
4. El API Gateway del borde empieza a devolver HTTP 504.

Lo que ve el usuario, y lo que dispara la alerta, es el paso 4. Lo que hay que arreglar es el 1. Miren el paso 3: el *retry* que en clases anteriores era la solución, acá es el combustible. Volveremos sobre esto ...

El Core Analysis Loop

Análisis por primeros principios: cada paso usa una señal distinta.

1. **Observar la anomalía agregada** $\rightarrow$ **Métrica**. El p99 saltó. Sabés *que* algo pasa, no *qué*.

El Core Analysis Loop

Análisis por primeros principios: cada paso usa una señal distinta.

1. **Observar la anomalía agregada** → **Métrica**. El p99 saltó. Sabés *que* algo pasa, no *qué*.
2. **Diferenciar dimensiones** → **Eventos anchos**. ¿Qué atributo separa las solicitudes que fallaron de las que no? Por ejemplo, `availability_zone=us-east-1a` aparece en el 98 % de los fallos y en el 15 % del resto. Esto es el *soft context* haciendo el trabajo pesado.

El Core Analysis Loop

Análisis por primeros principios: cada paso usa una señal distinta.

1. **Observar la anomalía agregada** → **Métrica**. El p99 saltó. Sabés *que* algo pasa, no *qué*.
2. **Diferenciar dimensiones** → **Eventos anchos**. ¿Qué atributo separa las solicitudes que fallaron de las que no? Por ejemplo, `availability_zone=us-east-1a` aparece en el 98 % de los fallos y en el 15 % del resto. Esto es el *soft context* haciendo el trabajo pesado.
3. **Confirmar con una traza individual** → **Tracing**. Abrís una solicitud representativa y ves en qué span se fue el tiempo.

El Core Analysis Loop

Análisis por primeros principios: cada paso usa una señal distinta.

1. **Observar la anomalía agregada** → **Métrica**. El p99 saltó. Sabés *que* algo pasa, no *qué*.
2. **Diferenciar dimensiones** → **Eventos anchos**. ¿Qué atributo separa las solicitudes que fallaron de las que no? Por ejemplo, `availability_zone=us-east-1a` aparece en el 98 % de los fallos y en el 15 % del resto. Esto es el *soft context* haciendo el trabajo pesado.
3. **Confirmar con una traza individual** → **Tracing**. Abrís una solicitud representativa y ves en qué span se fue el tiempo.

Si el atributo que encontraste no explica lo suficiente, volvés al paso 2 con el subconjunto ya filtrado. Eso es lo que lo hace un *loop* y no una lista de pasos.

Majors et al., *Observability Engineering* (O'Reilly, 2022), cap. 8.

Modelo del Curso: Observabilidad en $D = (P, F, G, S, T)$

P (Problema): Entender y diagnosticar fallas imprevistas en producción.

F (Factores): Clock drift, cascadas de fallas, latencia de red, cardinalidad.

G (Garantías): Reconstrucción de causalidad y aislamiento de causas raíz.

S (Solución): Trenza única (OTLP) + Hard Context (W3C) + Instrumentación en 2 capas + Core Analysis Loop.

T (Trade-offs): Overhead de cómputo y red por telemetría; costo de retención y necesidad de sampling.

Cierre y Conexión con Clase 11

Idea central de cátedra

Observabilidad distribuida no es “mirar dashboards ni abrir tres pestañas”. Es diseñar el software para que transporte contexto causal y pueda explicarse a sí mismo cuando falle.

Cierre y Conexión con Clase 11

Idea central de cátedra

Observabilidad distribuida no es "mirar dashboards ni abrir tres pestañas". Es diseñar el software para que transporte contexto causal y pueda explicarse a sí mismo cuando falle.

Hacia la próxima clase: ¿Y si el sistema falla igual?

Si ya podemos observar y diagnosticar que un servicio downstream se degradó... ¿cómo evitamos que arrastre a todo el sistema?

→ **Clase 11: Resiliencia distribuida (Circuit Breakers, Bulkheads y Backoff).**